

EdgeP4: In-Network Edge Intelligence for a Tactile Cyber-Physical System Testbed Across Cities

Nithish Krishnabharathi Gnani, Joydeep Pal, Deepak Choudhary, Himanshu Verma,
Soumya Kanta Rana, Kaushal S Mhapsekar¹, T V Prabhakar, and Chandramani Singh

Indian Institute of Science, Bengaluru, India ¹ North Carolina State University, Raleigh, USA

{nithishgnani, joydeepal, deepakcl, himanshuv, soumyarana, tvprabs, chandra}@iisc.ac.in, ¹kmhapse@ncsu.edu

Abstract—Tactile Internet based operations, e.g., telesurgery, rely on the human operator for end-to-end closed loop control to achieve accuracy. While feedback and control are subject to network latency and packet loss, critical operations such as pose correction, fiducial marking, tremor reduction, etc. are tasks that should be carried out automatically and thus dispensing with these feedback signals from traversing the network to the other end. We design two edge intelligence algorithms hosted at P4 programmable end switches placed 400 km apart. These algorithms locally compute and command corrective signals. We implement these algorithms entirely on data plane on Netronome Agilio SmartNICs. “pose correction” is placed at the edge switch connected to an industrial robot gripping a tool. The round trip between transmitting force sensor array readings to the edge switch and receiving correct tip coordinates at the robot is shown to be less than 100 μ s. “tremor suppression” is placed at the edge switch connected to the operator. It suppresses physiological tremors, which improves the application’s performance and also reduces the network load up to 99.9%. Our EdgeP4 framework allows edge intelligence modules to seamlessly switch between the algorithms based on the tasks being executed by devices connected to their ports.

Index Terms—Cyber-Physical Systems, Edge Intelligence, P4, SmartNIC, Tactile Internet, Teleoperation

I. INTRODUCTION AND MOTIVATION

Grip inspection is an essential requirement for robotic arms with fingered grippers. Imagine a surgeon gripping a surgical tool. They position the robot, close the gripper, and receive feedback on the “tool centre point” pose. If the grip is unsatisfactory, they can adjust and re-grip, iterating until a firm hold is achieved. In traditional robot-assisted surgery and diagnostics, surgeons directly control the robot, like in the DaVinci System [1]. This colocation constraint can be overcome by Time-Sensitive Networking (TSN) which offers bounded latency and reliability over longer distances, eliminating cyber-sickness concerns [2].

Edge intelligence systems can autonomously refine gripper actions while providing real-time feedback, freeing human operators to perform core application-related tasks. Interestingly, artificial intelligence (AI) and machine learning (ML) powered grip inspection often require multiple time-synchronized camera setups for simultaneous snapshots to provide complete visibility when the gripper engulfs the tool. Furthermore, image processing algorithms are used to evaluate the grip and calculate corrective pose data. GPU-based embedded modules assist in meeting the millisecond latency requirements of tac-

tile Cyber-Physical Systems (CPS) by running edge intelligent CPS algorithms.

Traditional edge infrastructure struggles to meet the ultra-fast response times needed for simultaneous real-time kinematic control of multiple CPS devices [3]. In this work, we implement intelligence at the switch ports in a tactile CPS, as shown in Fig. 1. Our testbed, spanning 400 km across two cities, consists of TSN-capable switches built using P4 programmable Smart Network Interface Cards (SmartNIC) and CPS devices such as a haptic device, and a robotic arm equipped with a two-fingered gripper. The SmartNICs run edge intelligence algorithms. The ports are programmed to switch at line rate, between several algorithms required by the CPS device connected to it. This eliminates the need for dedicated embedded controllers or specialized edge intelligence modules for each device. For grip correction, traditional AI-based approaches using image data from multiple cameras can be computationally expensive and require data communication and time-synchronization, posing challenges for processing at switch ports. In contrast, our solution leverages force sensor arrays for grip inspection, requiring only a press against the tool for grip detection with no synchronization requirement. An algorithm built for sensor data analysis and calculating pose correction resides on *Edge Switch 2* (Fig. 1) connected to the robot autonomously performs the gripping activity. Furthermore, *Edge Switch 1* intelligently masks human operator hand tremors from reaching the robot, enhancing performance and reducing network load. Our contributions are as follows:

- Designing and implementing an algorithm on the edge switch port that monitors the grip and provides pose correction to firmly hold a tool in position.
- Designing and implementing an intelligent hand tremor detection algorithm on the edge switch port to reduce its effect on the task performed by the robot as well as to reduce the network traffic.
- EdgeP4 - a framework for switch ports to detect and automatically switch between several algorithms specific to a task.
- Demonstrating and evaluating EdgeP4 in a Tactile CPS testbed setup across 400 km distance.

II. RELATED WORK

Laki et al. [3] propose offloading real-time velocity control of a robotic arm from the cloud to a Tofino switch [4],

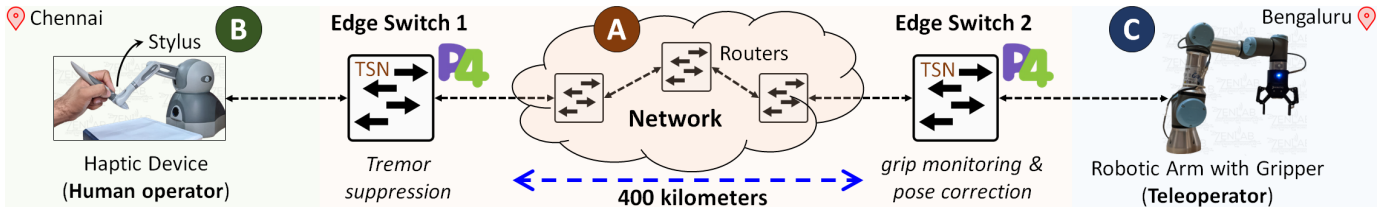


Fig. 1. Tactile CPS Testbed: Human operator uses a haptic device to control a robot 400 km away via P4-programmable edge switches

improving latency and flexibility for specific robot operations. Their implementation leverages “URcap” scripts tailored for Universal Robots’ “e-series” UR5 robots. Our work complements this vendor specific solution by introducing a more general P4-based edge intelligence framework for Tactile CPS to lessen human cognitive burden by automating tasks like precise gripping and alleviating from hand tremors. Furthermore, it improves control loop latency and reduces network load by analysing both control and feedback data.

Existing P4 applications of match-action and MicroC primarily focus on data packet inspection and offline AI model inference. While Xiong et al. [5] map pre-trained models for in-network classification, and Langlet et al. [6] infuse ANN inference into the P4 data plane, these approaches rely on pre-defined models. Large ML models not only introduce latencies, they also require a significant amount of training data. In our work, the switch port runs our geometry based pose correction algorithm under in-situ real-time conditions.

Existing robotic grasping solutions often rely on vision, tactile information, and fuzzy logic [7], [8], [9]. While tactile sensors can detect slip, pose correction is accomplished using multiple camera systems. This introduces uncertainty in achieving a stable grip and the need for cumbersome camera setups. Additionally, while most grippers control output force [10], there is a lack of force sensor array solutions for estimating accurate tool grip during real-time teleoperation.

Physiological tremor, characterized by fine hand and finger shaking, is often imperceptible [11]. While typically starting at 8-9Hz with 100 μ m amplitude, they can increase over time, reaching 2mm at 4-6Hz [12]. There have been significant developments focused on tremor detection and modeling [13], [14], with some applications in microsurgical instruments [15], [16]. These instruments used directly on patients, employ aggressive filtering algorithms and piezoelectric actuators to counteract tremors (e.g., Riviere et al.’s WFLC achieving 69% tremor suppression [16]). In our work, we not only apply threshold based tremor detection but also suppress tremor data transmission. This eliminates the need for additional actuators on the surgical tools.

III. SYSTEM ARCHITECTURE

A. Testbed Description

Fig. 1 shows our inter-city testbed spanning 400 km between Bengaluru and Chennai, India. In this work, we utilise the testbed to study the feasibility and performance of running edge intelligence algorithms on switch ports for Tactile CPS

applications like robotic manipulation. Our testbed comprises of three domains. Network Domain (A) handles packet communication between haptic devices and robots using TSN-capable switches (*Edge Switch 1 & 2*) with P4-programmable Netronome NFP-4000 Flow Processor [17] based SmartNICs. These support Time Aware Shaper (TAS) for bounded latencies in scheduled traffic (e.g., haptic data). Expedited forwarding prioritizes haptic and control packets on the Wide Area Network (routers) connecting the cities. In the Human Operator Domain (B), a haptic device (Geomagic Touch [18]) is manipulated by the human operator. The device includes the “OpenHaptics toolkit” software and C-programming tools are used to acquire its kinematic data. This data is communicated to the Teleoperator Domain (C) in which a UR3 robot (CB version) from Universal Robots [19] is equipped with a Robotiq 2F-85 gripper [20]. Python serial library is used to control the gripper. A maximum force of 235 *N* may be applied to grasp an object. A custom-built 5 $\times$ 3 force sensor array with custom fingertips is attached to the gripper (Fig. 2). UR3 is designed for factory settings with a fixed set of tasks that can be programmed using their proprietary PolyScope interface host computer. To achieve real-time control, we leverage the robot’s ability to be externally controlled by a computer running Robot Operating System (ROS) with Python programming in conjunction with PolyScope. We use the *CartesianTrajectoryController* from the ROS driver’s *rospy* library to command robot movements. By design, this ROS based software controller could run on any computer in the network. Robot pose and force sensor data are provided as feedback to Domain B. As a human operator manipulates the haptic device stylus, the teleoperator robot mirrors the stylus motion, and this is displayed to the operator via a video feed. Open and close commands to the gripper are sent using buttons on the stylus.

As an example demonstration, the task we have chosen to perform is to grip a tool using the gripper on the robot and to move it along paths resembling specific shapes. In this work, we consider a rigid tool with a handle grip consisting of a flat surface. We restrict the applied force for safety. The grip is considered good if it is held firmly along the axis over its length. The two-finger gripper’s orientation is manipulated only in a plane parallel and equidistant to the gripping surfaces of the fingers.

B. Building a Force Sensor Array

To enable object grasping with the two-fingered gripper with force and contact region measurement, we built a custom 5 $\times$ 3

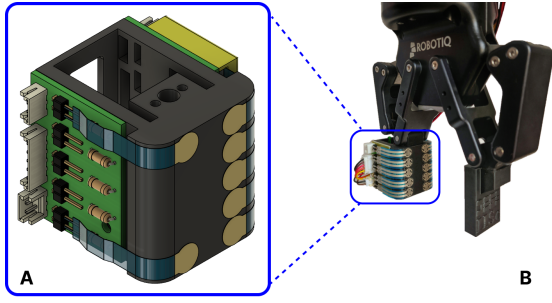


Fig. 2. A. CAD render of modified gripper fingertips. B. 3D printed custom fingertip equipped with a force sensor array attached to the gripper.

force sensor array for fingertips. Leveraging the affordability and availability of Force-Sensitive Resistors (FSRs), this array provides uniform contact surface coverage. Measured data from the FSRs feeds into the pose correction algorithm described in Section IV-A. Each FSR output connects to a 10-bit ADC input on an ATmega2560 microcontroller. The controller's internal voltage reference is enabled. The circuitry is made using a custom-built PCB with a footprint of $30\text{ mm} \times 40\text{ mm}$. Digital output from the controller is communicated to the robot host over a serial port. With 75% of the gripper's maximum force applied, the output reaches approximately 600.

The stock Robotiq 2F-85 gripper fingertips were replaced with a custom design (Fig. 2A). We integrated the custom-built PCB within the design to make it compact. This CAD-designed and 3D-printed fingertip facilitates easy FSR replacement and accommodates the PCB and cables without compromising gripper functionality. Figure 2B showcases the integrated force sensor array mounted onto the gripper.

C. Standardizing Data Exchange

In our work, it is crucial to establish efficient reading and manipulation of data using C, Python and P4 across the three domains (Fig. 1). P4 targets are built to process packet headers in real time and are generally not designed to work with their payload due to high resource requirements. However, to enable edge intelligence on switch ports, there is a need to modify the coordinate data in the payload. In our approach, we achieve payload data manipulation in P4 by maintaining a specific structure in which the data is packaged across all the devices in the three domains. This is done by declaring a custom header called *coordinate_metadata* which corresponds to the structure of the payload in P4.

TABLE I
coordinate_metadata HEADER

SID	TID	x	y	z	qx	qy	..	b1	b2	f0	..	f14
-----	-----	---	---	---	----	----	----	----	----	----	----	-----

Table I shows the *coordinate_metadata* header, made up of different 16-bit fields. The data is sent using user datagram protocol (UDP) over TSN-enabled ethernet. The relevant data from any of the domains are inserted into various fields of

coordinate_metadata. Each UDP packet with the data of the haptic device has a unique sequence ID (SID). The task ID (TID) field is used to track different tasks and is explained in section IV-C. The coordinates of the input haptic device are sent using x,y and z fields with sign information. The button information is sent using b1 and b2 fields. Feedback data contains the pose of the robotic arm, which comprises position and quaternion orientation, which is contained in the fields x, y, z, qx, qy, qz, and qw. The force sensor data is sent using the fields f0 - f14. This feedback data is used by the switch ports for pose correction. A telemetry header is also sent with the packet giving the ingress and egress timestamps, the packet length, and the ingress and egress ports. These attributes are used in performance evaluation. The total size of each ethernet frame is 130 bytes.

The byte-ordering scheme of the Intel processors in the cloud computer running ROS and the haptic device system is Little-Endian, while that of the SmartNIC is Big-Endian. Therefore, we convert each 16-bit field in *coordinate_metadata* into Big Endian before transmitting it to the switch. Since P4 does not have floating-point numbers as a built-in base type [21], to transmit position and quaternion data from Domains B and C to A, we multiply them by appropriate exponents of ten and convert the fields to signed integers. We divide the received data by the same multiplier. The multipliers are chosen such that the position resolutions of the haptic device and the robot are not constrained by data-type conversion. The computation latency for this operation falls well within tactile CPS requirements, as shown in Section V.

IV. EDGE INTELLIGENT SWITCH PORTS

A. Automatic Pose Correction

Algorithm 1 outlines the pose correction method, which computes the robot's translation and rotation steps required for accurate gripping of the tool. This is a one-time calibration step for a specific tool and generates a baseline table of pose corrections for all possible orientations. The pose correction output is stored as a match action table in a *json* file which is used by the P4 program running on the edge switch ports performing automatic pose correction. The corrected pose is obtained by calculating the translation distance and angle using basic cartesian coordinate arithmetic. This is demonstrated in Fig. 3. In Fig. 3A, to begin with, the robot's gripper finger is positioned such that the tool is placed at the bottom at a certain angle. It presses against the force sensors 10 and 14, and for this position, Fig. 3B shows the distance along the X and Y axes and the angle along which the robot has to move in order to achieve the correct orientation of the gripped tool. These distances and the angle are indicated by $dist_x$, $dist_y$ and $angle$, respectively. The algorithm forms a list of all the combinations of sensor pairs and calculates $dist_x$, $dist_y$ and $angle$ with respect to Sensor 2 (shaded in green). The final gripper position after pose correction is shown in Fig. 3C. When the tool is held in the correct position, the readings of the FSRs in the middle column will cross the threshold. If the

gripper is closed without holding the tool, all the FSR readings exceed the threshold. These conditions are stored in the output table.

Algorithm 1 Pose correction calculation

Input: n, m, gap_x, gap_y

Output: correction table

- 1: Create an array of $n \times m$ FSRs
/* Sensor array has $n \times m$ FSRs spaced apart by gap_x and gap_y in horizontal and vertical axes (Fig. 3A) */
 - 2: For every FSR pair, execute steps 3 & 4
/* Example calculations below for cells 10 and 14 (Fig. 3B) */
 - 3: $dist_x = (x_{10} - x_2) \times gap_x$; $dist_y = (y_{10} - y_2) \times gap_y$;
/* Calculate distance between base cell 2 (green) and chosen cell */
 - 4: $angle = (x_{10} - x_2) \times gap_x / (y_{10} - y_2) \times gap_y$
/* Calculate the angle between the two FSRs and convert to quaternions qx, qy, qz, qw using Euler angles to quaternions conversion formula */
 - 5: Store $dist_x, dist_y, quaternions (qx, qy, qz, qw)$ in a correction table
-

TABLE II
EXAMPLE FORCE SENSOR ARRAY READING

FSR array	3	5	..	8	20	720	12	0	1	760	3
Index	0	1	..	7	8	9	10	11	12	13	14

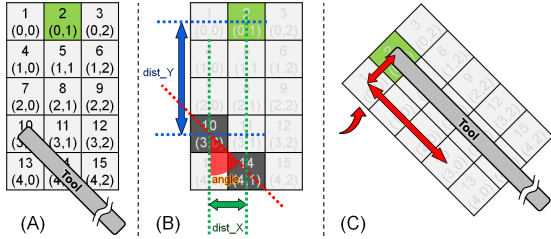


Fig. 3. An illustration of pose correction

Fig. 4 shows the entities in the network, including the haptic device, the teleoperator robotic arm and the *Edge Switch 2*, which is accessible to the robot. It illustrates the flow of control, feedback and pose correction packets. The haptic device originates control packets, and the egress port of *Edge Switch 2* adds telemetry data to these before forwarding these to the teleoperator. Algorithm 2 describes the implementation of grip inspection and pose correction intelligence in the switch port using P4. Feedback packets containing the pose of the robot and readings of the force sensor array in *coordinate_metadata* header are inspected by the ingress port of *Edge Switch 2*. The P4 program begins by parsing the *coordinate_metadata* header. It then executes Algorithm 3, an extern function developed in MicroC which identifies the sensors in the force sensor array exceeding a predefined threshold of 500. The extern function also stores the indices of the first and the last among the identified sensors in registers named *se* and *es*. Table II gives

a snapshot of the *coordinate_metadata* header corresponding to a force sensor array reading where *se* is 9 and *es* is 13. If *es* is greater than *se*, indicating an incorrect grip, *Edge Switch 2* clones the packet before forwarding it to the haptic device. The algorithm retrieves the necessary data ($dist_x, dist_y, quaternions$) from the match action table (MAT) of precalculated pose corrections using the key (*se, es*). These corrections are updated in the *coordinate_metadata* header of the cloned packet and the corrected packet is sent back to the teleoperator. The subsequent feedback packet from the teleoperator is inspected and the TID field in *coordinate_metadata* header is used to indicate whether the grip is correct or not, and the haptic device uses this information to notify the human operator. During the regular operation of the teleoperator, the feedback packet is forwarded to the haptic device without cloning.

Algorithm 2 Grip inspection and pose correction implementation in P4

Input: Teleoperator feedback (Fig. 4) with *coordinate_metadata* header

Output: a. Forward packet to haptic device (feedback)

- b. Send packet with corrected pose in *coordinate_metadata* to robot
 - 1: Parse *coordinate_metadata* header
 - 2: Run extern function *EdgeSensors* (Algorithm 3)
 - 3: **if** ($es \leq se$) **then**
 - 4: forward packet to haptic device
 - 5: **end if**
/* No incorrect grip detected, so send feedback packet to haptic device */
 - 6: **if** ($es > se$) **then**
 - 7: Clone the packet
 - 8: Forward original packet to haptic device
 - 9: $(dist_x, dist_y, quaternions) = MAT(se, es)$
 - 10: $x = x + dist_x$; $y = y + dist_y$; update quaternions
/* Update pose in *coordinate_metadata* */
 - 11: Forward correction packet to robot
 - 12: **end if**
/* Incorrect grip, so clone and send corrections to robot; send feedback to haptic device */
-

Algorithm 3 *EdgeSensors* extern function in MicroC

Input: *FSR, threshold*

Output: End indices of sensors in the array that exceed threshold stored in registers *se, es*

- /* Example: index 9 & 13 in Table II corresponds to cells 10 & 14 in Fig. 3B */
 - /* *se*: start to end; *es*: end to start */
 - 1: **for** $i = 0$ to 14 **do**
 - 2: $se = i$
 - 3: **if** $FSR[i] > threshold$ **then**
 - 4: break
 - 5: **else**
 - 6: $i++$
 - 7: **end if**
 - 8: **end for**
 - 9: Similarly get *es* by looping in reverse
 - 10: Store *se* & *es* in register memory
-

B. Tremor suppression

The hand tremors of the operator while manipulating the haptic device translate into vibration of the teleoperator. We use a tremor suppression algorithm implemented in P4 to prevent the haptic packets associated with tremors from being sent to the teleoperator. This is described in Algorithm 4 which is executed directly on the ingress port of *Edge Switch 1* (Fig. 1). The algorithm begins with storing the initial 3D coordinates contained in the haptic data packet in the register memory. The coordinates in the subsequent haptic packet are compared to the stored values. If the L_1 -distance between these coordinates is greater than a threshold, the packet is forwarded and the new coordinates are stored, else the latter haptic data packet is discarded.¹ P4 does not have native support to compare negative numbers. Therefore, to manage the direction of tremors, we have developed a MicroC function supported directly on the switch port that converts negative values to absolute values.

Algorithm 4 Tremor suppression algorithm in P4-MicroC

Input: *thresh*, *x*, *y*, *z* in *coordinate_metadata* in packets from haptic device

Output: Conditionally accept or discard haptic data packets

```

1: (diffx ← x - oldx); (diffy ← y - oldy); (diffz ← z - oldz);
2: if (diffx >> 15 == 1) then
3:   diffx ← not(diffx) + 1;
4: end if
   /* Handling negative numbers - get absolute
   value from 2's complement representation */
5: Do the above for diffy and diffz
6: if diffx + diffy + diffz > thresh then
7:   oldx ← x; oldy ← y; oldz ← z;
   /* Update last stored coordinates */
8:   accept the packet
   /* Intentional motion ⇒ forward data */
9: else
10:  drop the packet
11: end if
    
```

C. Seamless Switching Between Algorithms

We use the 16 bit TID in the *coordinate_metadata* header not only to identify the capability of the robot but also to switch between different edge intelligence algorithms associated with different tasks. This framework facilitates edge intelligence algorithms to be offloaded to an edge switch that might connect to multiple robots. The necessary parameters linked to various tasks and robots are stored in the match action table of the switch. For example, *dist_x*, *dist_y* and *quaternions* are stored for a pose correction task, whereas the *threshold* is stored for a tremor suppression task. When a robot connected to a particular switch port is required to perform a different task or is replaced with another robot, the new task's TID is used as a key, and the switching process is transparent to the end user. The robot is automatically assigned tasks without reprogramming the switch. The parameters can be changed in runtime by updating the match action table.

¹More prevalent Euclidean distances are not used because P4 does not support *squaring*.

Thus, P4-based edge intelligent switches can identify and track all connected robots and associated tasks, making the proposed solution highly flexible and scalable.

V. EXPERIMENTAL RESULTS AND DISCUSSION

Fig. 4 captures the time taken by the packets across different events in the edge intelligent switch port running the automatic pose correction algorithm. In conventional pose correction of the robot, the feedback packets are inspected, corrections are calculated at the human operator end, and the control packets are sent back over the inter-city network. This latency is measured to be 10.7 *ms* on average. In contrast, the time duration between sending a feedback packet from the robot and receiving the corresponding pose correction packet using the edge intelligent switch port is observed to be less than 100 μ s. Thus, we not only achieve latency suitable for tactile CPS, but we also safeguard against indefinitely stalling the pose correction and feedback process due to packet losses across the network. We include the ingress and egress timestamps of packets into the telemetry header. The difference between the two is used to measure the time a packet spends inside the switch. Our experiments reveal that adding timestamps and port information requires a maximum of 120 *ns*. Furthermore, conditionally cloning a feedback packet and updating it with pose correction requires a maximum of 340 *ns*.

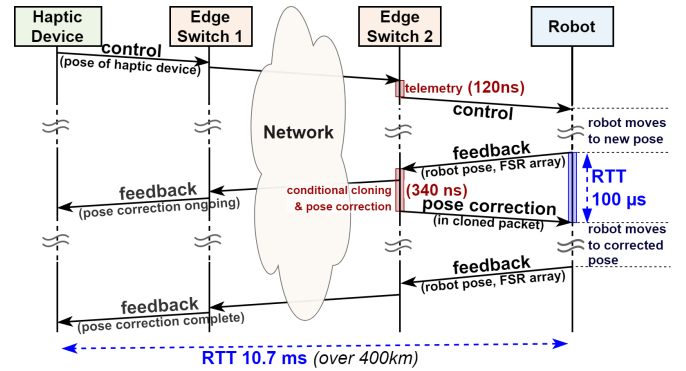


Fig. 4. Automatic pose correction - packet flow and latency measurements

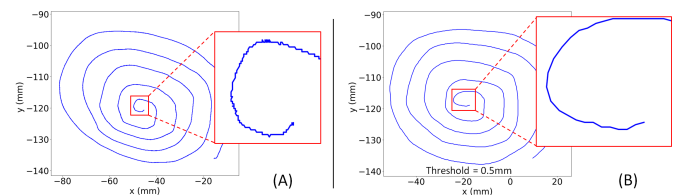


Fig. 5. (A) Tremor reflected in output movement; (B) Smooth movement achieved using tremor suppression with a threshold of 0.5mm

We demonstrate our tremor suppression algorithm by executing standard tasks such as a. Drawing a spiral using the stylus of the haptic device b. Drawing a straight line and c. Holding the stylus steadily [11]. We record the coordinates in the received packets at the robot. We experiment with the thresholds of 0.1 *mm*, 0.5 *mm*, 1 *mm* and 2 *mm*. The effect of the tremor suppression algorithm can be seen in Fig. 5. Fig. 5A shows the output movement of the teleoperator in the

absence of the algorithm. The hand tremors captured by the stylus of the haptic device are clearly visible in the form of the zig-zag motion of the teleoperator. Fig. 5B shows the smooth output resulting from the application of our Algorithm 4.

Tactile applications typically require control data to be sent at 1 *KHz*. Our haptic device provides a 1 *KHz* signal with a minimum amplitude of 55 μm . Since the packet size is 130 *bytes* including the *coordinate_metadata* header, this results in a data transmission rate of about 1 *Mbps*. The tremor suppression algorithm with a threshold of 0.5 *mm* for the three tasks results in a reduced data rate as low as 55 *Kbps* with a bandwidth savings of over 94%. Table III captures the number of packets discarded and the overall savings in bandwidth. Since drawing a spiral is more involved compared to drawing a straight line or holding the stylus, it warrants a higher number of packets to be transmitted. Consequently, the hold task has the maximum savings in bandwidth among the three. These tasks were performed in 15, 5 and 10 seconds, respectively. The effect of setting different thresholds while drawing a spiral is shown in Table IV. For example, for a 2 *mm* threshold, the data transmission rate can be as low as 15 *Kbps*. Furthermore, the last column shows that by suppressing tremors, we also reduce the total length travelled by the tool of the teleoperator. In a robot-assisted medical task such as making an incision, this can lead to reduced tissue bruising.

TABLE III
TRAFFIC REDUCTION FOR DIFFERENT TASKS

Task	No. of packets transmitted	No. of packets discarded	Average data rate (kbps)	Traffic reduction (%)
Spiral	813	14182	55	94.58
Line	250	4718	51	94.97
Hold	5	9950	0.5	99.95

TABLE IV
EFFECT OF DIFFERENT THRESHOLDS IN DRAWING A SPIRAL

Threshold (mm)	No. of packets transmitted	No. of packets discarded	Average data rate (kbps)	Traffic reduction (%)	Drawing length (mm)
None	14992	0	1015	0.0	600
0.1	2299	12621	148	84.6	507
0.5	813	14182	55	94.6	527
1	423	14494	29	97.2	506
2	226	14714	15	98.5	512

VI. CONCLUSION AND FUTURE WORK

We have presented two hardware implementation based edge intelligence algorithms - *pose correction* and *tremor suppression* - running on P4 programmable switch ports using SmartNICs over an inter-city Tactile CPS testbed. The experimental results in Section V vindicate that in-network intelligence can substantially reduce control loop latency to microseconds, enabling tactile applications and reducing network load. Further, intelligence for multiple networked devices is enabled by hosting several algorithms on the same edge

switch, seamlessly switching between them based on task needs, thus eliminating dedicated embedded modules.

Our future work entails designing and implementing more advanced edge intelligence algorithms, particularly AI-powered ones, for diverse tasks in Tactile CPS.

ACKNOWLEDGMENTS

Partly supported by the MeitY, Government of India (SP/MITO-20-0006) and the Centre for Networked Intelligence (a Cisco CSR initiative) at Indian Institute of Science.

REFERENCES

- [1] C. Freschi, V. Ferrari, F. Melfi, M. Ferrari, F. Mosca, and A. Cuschieri, "Technical review of the da vinci surgical telemanipulator," *The International Journal of Medical Robotics and Computer Assisted Surgery*, vol. 9, no. 4, pp. 396–406, 2013.
- [2] K. Polachan, J. Pal, C. Singh, and T. Prabhakar, "Assessing quality of control in tactile cyber-physical systems," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, 2022.
- [3] S. Laki, C. Györgyi, J. Pető, P. Vörös, and G. Szabó, "{In-Network} velocity control of industrial robot arms," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022.
- [4] Intel® Tofino™, Intel. [Online]. Available: <https://bit.ly/Intel-Tofino>
- [5] Z. Xiong and N. Zilberman, "Do switches dream of machine learning? toward in-network classification," in *Proceedings of the 18th ACM workshop on hot topics in networks*, 2019, pp. 25–33.
- [6] J. Langlet, A. Kassler, and D. Bhamare, "Towards neural network inference on programmable switches," in *2nd P4 Workshop in Europe (EUROP4)*, Cambridge UK., 2019.
- [7] J. J. van Vuuren, L. Tang, I. Al-Bahadly, and K. Arif, "Towards the autonomous robotic gripping and handling of novel objects," in *2019 14th IEEE Conference on Industrial Electronics and Applications (ICIEA)*. IEEE, 2019, pp. 1006–1010.
- [8] R. Sadeghian, S. Shahin, and S. Sareh, "Vision-based self-adaptive gripping in a trimodal robotic sorting end-effector," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2124–2131, 2022.
- [9] F. Veiga, J. Peters, and T. Hermans, "Grip stabilization of novel objects using slip prediction," *IEEE transactions on haptics*, vol. 11, no. 4, 2018.
- [10] *Does the 2F gripper have force sensing*, DoF, a Robotiq Community. [Online]. Available: <https://dof.robotiq.com/discussion/207>
- [11] J. Alty, J. Cosgrove, D. Thorpe, and P. Kempster, "How to use pen and paper tasks to aid tremor diagnosis in the clinic," *Practical Neurology*, vol. 17, no. 6, pp. 456–463, 2017.
- [12] R. N. Stiles, "Frequency and displacement amplitude relations for normal hand tremor," *Journal of applied physiology*, vol. 40, no. 1, pp. 44–54, 1976.
- [13] K. C. Veluvolu, W. T. Latt, and W. T. Ang, "Double adaptive bandlimited multiple fourier linear combiner for real-time estimation/filtering of physiological tremor," *Biomedical Signal Processing and Control*, vol. 5, no. 1, pp. 37–44, 2010.
- [14] J. Zhang, F. Chu, and N. Mohammed, "Dsp controller based signal processing of physiological hand tremor," in *Proceedings of the 2005, American Control Conference, 2005*. IEEE, 2005.
- [15] W. T. Ang, C. N. Riviere, and P. K. Khosla, "An active hand-held instrument for enhanced microsurgical accuracy," in *Medical Image Computing and Computer-Assisted Intervention—MICCAI 2000: Third International Conference, Pittsburgh, PA, USA, October 11–14, 2000. Proceedings 3*. Springer, 2000, pp. 878–886.
- [16] C. N. Riviere, W. T. Ang, and P. K. Khosla, "Toward active tremor canceling in handheld microsurgical instruments," *IEEE Transactions on Robotics and Automation*, vol. 19, no. 5, pp. 793–800, 2003.
- [17] *NFP-4000 Flow Processor*, Netronome SmartNIC. [Online]. Available: https://d3ncevyc0dfnh8.cloudfront.net/media/documents/PB_NFP-4000-7-20.pdf
- [18] *Touch Haptic Device*, 3D Systems. [Online]. Available: <https://www.3dsystems.com/haptics-devices/touch>
- [19] Universal Robots. [Online]. Available: <https://www.universal-robots.com/products/cb3/>
- [20] *Adaptive Grippers*, Robotiq. [Online]. Available: <https://robotiq.com/products/2f85-140-adaptive-robot-gripper>
- [21] *P4 Language Specification (V16)*, P4 Language Consortium, 2019. [Online]. Available: <https://p4.org/p4-spec/docs/P4-16-v1.0.0-spec.html>